Barramento CAN do Sistema AEB

Modelagem e Simulação em Simulink

Projeto:	Residência Tecnológica Stellantis/UFPE
Módulo:	Modelagem — Barramento CAN
Data:	Março de 2026
Norma:	ISO 11898 (CAN), AUTOSAR, ISO 26262 ASIL-B
Arquivo:	`AEB_CAN.slx`
DBC:	`can/aeb_system.dbc` — 500 kbps

Nota Técnica

Este documento descreve o modelo Simulink do barramento CAN do sistema AEB, com foco na codificação/decodificação de sinais conforme o arquivo DBC, no mecanismo de AliveCounter, no CRC-4 da mensagem ASIL-B e na injeção e detecção de falhas de comunicação.

Gerado com MATLAB R2025b + Simulink

Sumário

1 Introdução **2**

2 Barramento CAN — Visão Geral **2**
2.1 Configuração do barramento . . . 3
2.2 Temporização das mensagens . . . 3

3 Detalhamento das Mensagens **3**
3.1 AEB_BrakeCmd — 0x080 (ASIL-B) . . . 3
3.2 AEB_EgoVehicle — 0x100 . . . 4
3.3 AEB_RadarTarget — 0x120 . . . 4
3.4 AEB_FSMState — 0x200 . . . 4
3.5 AEB_Alert — 0x300 (evento) . . . 4

4 Codificação DBC **5**
4.1 Princípio de codificação . . . 5
4.2 Implementação no Simulink . . . 5

5 AliveCounter **5**
5.1 Mecanismo . . . 6
5.2 Simulação . . . 7

6 CRC de 4 Bits **7**

7 Injeção de Falhas **8**

8 Diagnóstico CAN **9**

9 Pipeline do Modelo Simulink **10**

10 Resultados de Simulação **11**
10.1 Sinais decodificados — cenário nominal . . . 11
10.2 Parâmetros DBC consolidados . . . 11
10.3 Uso . . . 11

11 Sincronização `v_target_cached_` **12**

12 Conclusão **12**

1 Introdução

O barramento CAN (*Controller Area Network*) é a espinha dorsal de comunicação do sistema AEB. No veículo real, nenhum módulo acessa diretamente os dados de outro: toda informação transita pelo barramento com temporização cíclica, fator de escala e mecanismos de integridade. A modelagem Simulink deste subsistema tem dois objetivos principais:

1. **Verificação de codificação**: confirmar que os fatores, offsets e tamanhos de bit definidos no arquivo DBC são aplicados corretamente em ambos os sentidos (TX e RX).

2. **Injeção e detecção de falhas**: validar que os mecanismos de AliveCounter e CRC-4 detectam corrupção de mensagem e perda de frame, gerando os DTCs correspondentes conforme ISO 14229 (UDS).

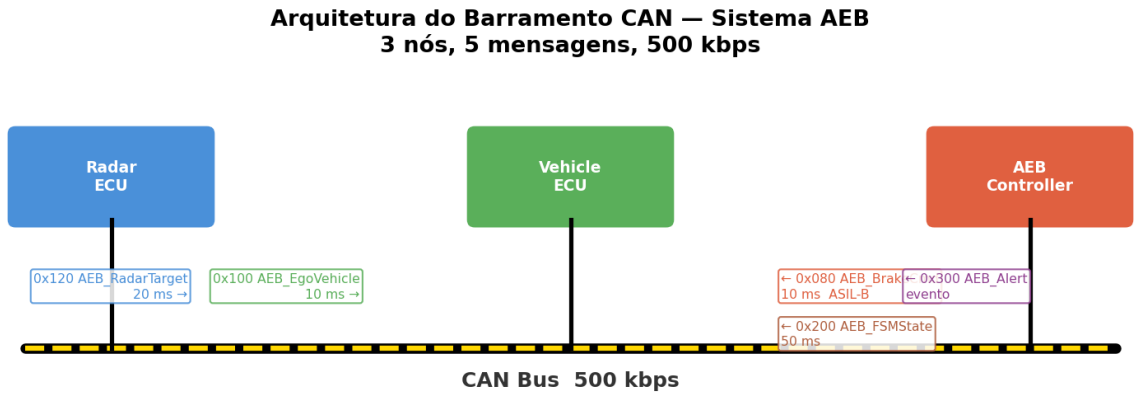

Figura 1: Arquitetura do barramento CAN do sistema AEB: 3 nós, 5 mensagens, 500 kbps.

Nota Técnica

O arquivo de banco de dados CAN `aeb_system.dbc` é a fonte de verdade formal para todos os parâmetros de sinalização. Qualquer discrepância entre o modelo Simulink e o DBC deve ser resolvida em favor do DBC.

2 Barramento CAN — Visão Geral

2.1 Configuração do barramento

Parâmetro	Valor
Velocidade	500 kbps
Nós	`AEB_Controller`, `Vehicle_ECU`, `Radar_ECU`
Mensagens	5 (cíclicas + evento)
Comprimento máximo	8 bytes (CAN clássico)
Norma	ISO 11898-1

2.2 Temporização das mensagens

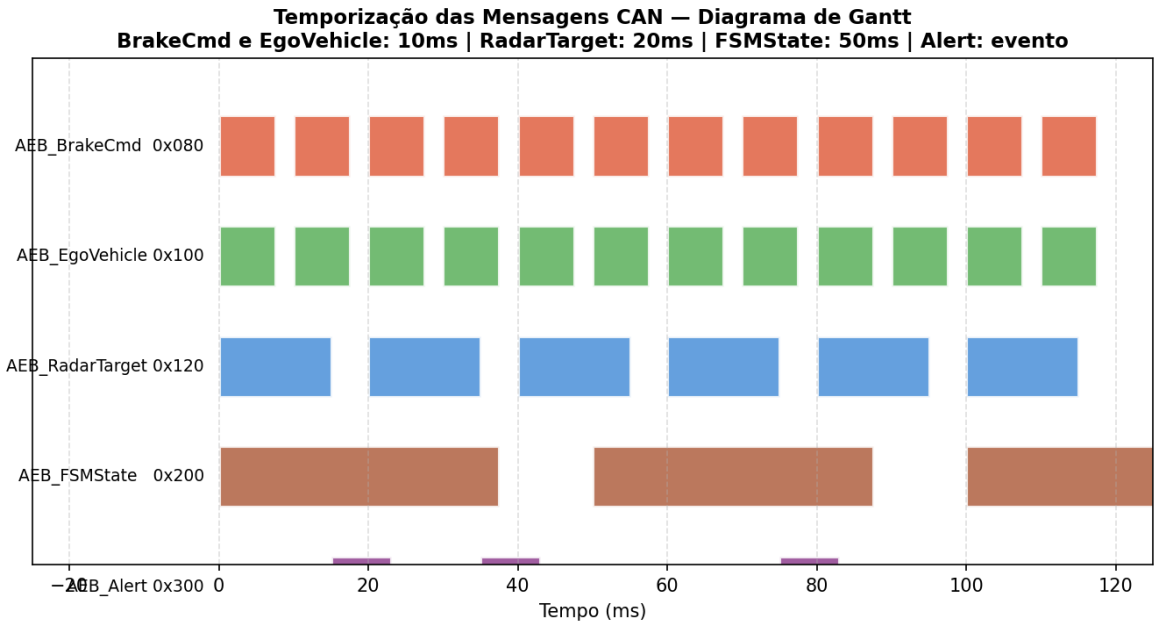

Figura 2: Diagrama de Gantt — temporização das 5 mensagens CAN. `AEB_BrakeCmd` e `AEB_EgoVehicle` a 10 ms; `AEB_RadarTarget` a 20 ms; `AEB_FSMState` a 50 ms; `AEB_Alert` dirigida por evento.

A Figura 2 evidencia uma consequência importante da assincronismo entre `AEB_EgoVehicle` (10 ms) e `AEB_RadarTarget` (20 ms): o controlador AEB deve manter `v_target_cached_`, calculado dentro do *callback* do radar, para garantir que velocidade do ego e velocidade relativa sejam contemporâneas no cálculo de TTC.

3 Detalhamento das Mensagens

3.1 AEB_BrakeCmd — 0x080 (ASIL-B)

Mensagem de segurança crítica: carrega o comando de frenagem do `AEB_Controller` para o atuador de freio. Protegida por AliveCounter e CRC de 4 bits.

```
BO_ 128 AEB_BrakeCmd:  4 AEB_Controller
SG_ BrakeRequest :  0|1@1+ (1,0) [0|1]  Vehicle_ECU
SG_ BrakePressure :  1|15@1+ (0.1,0) [0|3276.7] "bar"Vehicle_ECU
SG_ BrakeMode :  16|3@1+ (1,0) [0|7]  Vehicle_ECU
SG_ AliveCounter :  24|4@1+ (1,0) [0|15]  Vehicle_ECU
SG_ CRC : 28|4@1+ (1,0) [0|15]  Vehicle_ECU
```

Atenção ASIL-B

A mensagem `AEB_BrakeCmd` é classificada como ASIL-B. O receptor (`Vehicle_ECU`) deve rejeitar o frame imediatamente ao detectar CRC incorreto ou AliveCounter com diferença $\neq 1 \pmod{16}$. Falha no AliveCounter por >5 ciclos activa o DTC C1005; erro de CRC activa C1004.

3.2 AEB_EgoVehicle — 0x100

```
BO_ 256 AEB_EgoVehicle:  8 Vehicle_ECU
SG_ VehicleSpeed :  0|16@1+ (0.01,0) [0|655.35] "m/s"
SG_ LongAccel :  16|16@1+ (0.001,-32) [-32|33.535] "m/s^2"
SG_ YawRate :  32|16@1+ (0.01,-327.68) [-327.68|327.67] "deg/s"
SG_ SteeringAngle :  48|16@1+ (0.1,-3276.8) [-3276.8|3276.7] "deg"
```

A aceleração longitudinal usa *offset* de -32 m/s^2 para representar valores negativos em campo sem sinal: $\text{raw} = \lfloor(\text{físico} + 32)/0{,}001\rceil$.

3.3 AEB_RadarTarget — 0x120

```
BO_ 288 AEB_RadarTarget:  8 Radar_ECU
SG_ TargetDistance :  0|16@1+ (0.01,0) [0|655.35] "m"
SG_ RelativeSpeed :  16|16@1+ (0.01,-327.68) [-327.68|327.67] "m/s"
SG_ TTC : 32|16@1+ (0.001,0) [0|65.535] "s"
SG_ Confidence :  48|8@1+ (1,0) [0|15]
```

3.4 AEB_FSMState — 0x200

FSMState	Nome	Descrição
0	OFF	Sistema desligado
1	STANDBY	Aguardando ameaça
2	WARNING	Alerta ao motorista
3	BRAKE_L1	Frenagem leve: -2 m/s^2
4	BRAKE_L2	Frenagem moderada: -4 m/s^2
5	BRAKE_L3	Frenagem total: -6 m/s^2
6	POST_BRAKE	Manutenção pós-parada

3.5 AEB_Alert — 0x300 (evento)

Publicada apenas em mudanças de estado FSM. Comanda o HMI de alertas visuais (*BuzzerCmd*): SingleBeep (WARNING), DoubleBeep (L1), FastPulse (L2), Continu-

ousBeep (L3).

4 Codificação DBC

4.1 Princípio de codificação

O padrão DBC define para cada sinal: posição inicial em bits, comprimento em bits, endianness (`@1` = little-endian), sinalidade (`+` = sem sinal), fator e *offset*:

Fórmula DBC

$$\text{valor_físico} = \text{raw} \times \text{fator} + \text{offset} \tag{1}$$

$$\text{raw} = \left\lfloor \frac{\text{valor_físico} - \text{offset}}{\text{fator}} \right\rceil \tag{2}$$

Exemplo: `VehicleSpeed` $= 10{,}0$ m/s $\Rightarrow$ raw $= \lfloor 10{,}0/0{,}01 \rceil = 1000$.

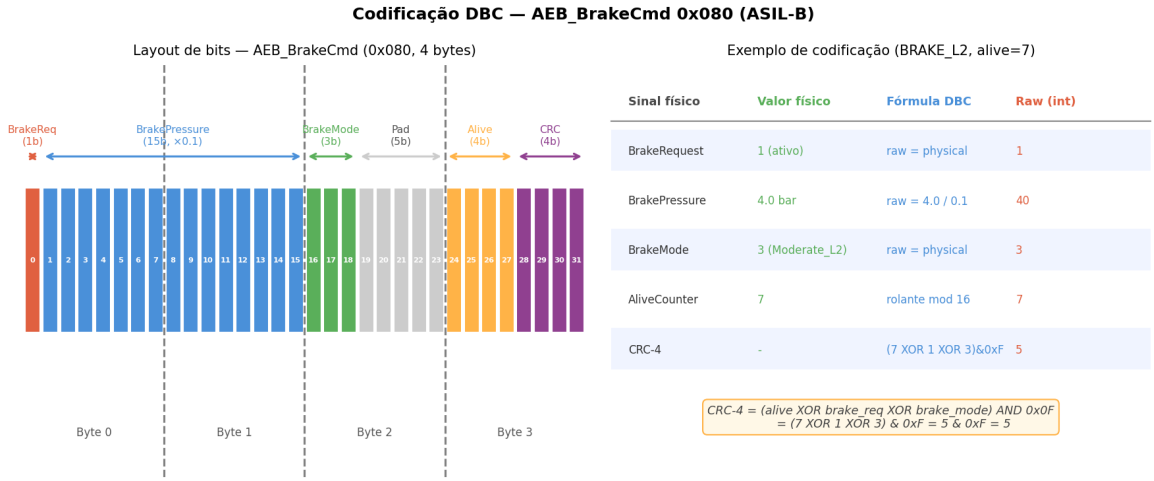

Figura 3: Layout de bits do frame `AEB_BrakeCmd` (4 bytes) e exemplo numérico de codificação para BRAKE_L2.

4.2 Implementação no Simulink

No bloco `TX_BrakeCmd`, o bloco MATLAB Function implementa:

Listing 1: Codificação DBC em MATLAB Function

```
% BrakeRequest: 1 bit, sem fator/offset
bc_req_r    = double(brake_req > 0);

% BrakePressure: 15 bits, fator 0.1, range 0-3276.7 bar
bc_press_r = double(min(round(brake_press / 0.1), 32767));

% BrakeMode: 3 bits (0=Off, 1=Warning, 2=L1, 3=L2, 4=L3)
bc_mode_r   = double(min(max(round(brake_mode), 0), 5));
```

5 AliveCounter

5.1 Mecanismo

O AliveCounter é um campo de 4 bits presente em `AEB_BrakeCmd` e `AEB_RadarTarget`. Incrementa a cada frame transmitido, reiniciando de 15 para 0 (contador rolante módulo 16):

AliveCounter

$$\text{alive}_{k+1} = (\text{alive}_k + 1) \bmod 16 \tag{3}$$

Sequência esperada: $0, 1, 2, \ldots, 14, 15, 0, 1, 2, \ldots$

O receptor detecta falha quando dois valores consecutivos não diferem por exatamente 1 módulo 16. Isso captura tanto **perda de frame** (diferença ≥ 2) quanto **travamento do transmissor** (diferença $= 0$).

Listing 2: AliveCounter no TX (MATLAB Function)

```
persistent alive_cnt;
if isempty(alive_cnt), alive_cnt = uint8(0); end
alive = double(alive_cnt);
alive_cnt = bitand(alive_cnt + uint8(1), uint8(15));  % mod 16
```

Listing 3: Verificacao do AliveCounter no RX

```
persistent last_alive;
if isempty(last_alive), last_alive = -1; end
if last_alive < 0
    alive_err = 0;  % primeiro frame: sem referencia
else
    alive_err = double(alive ~= mod(last_alive + 1, 16));
end
last_alive = alive;
```

5.2 Simulação

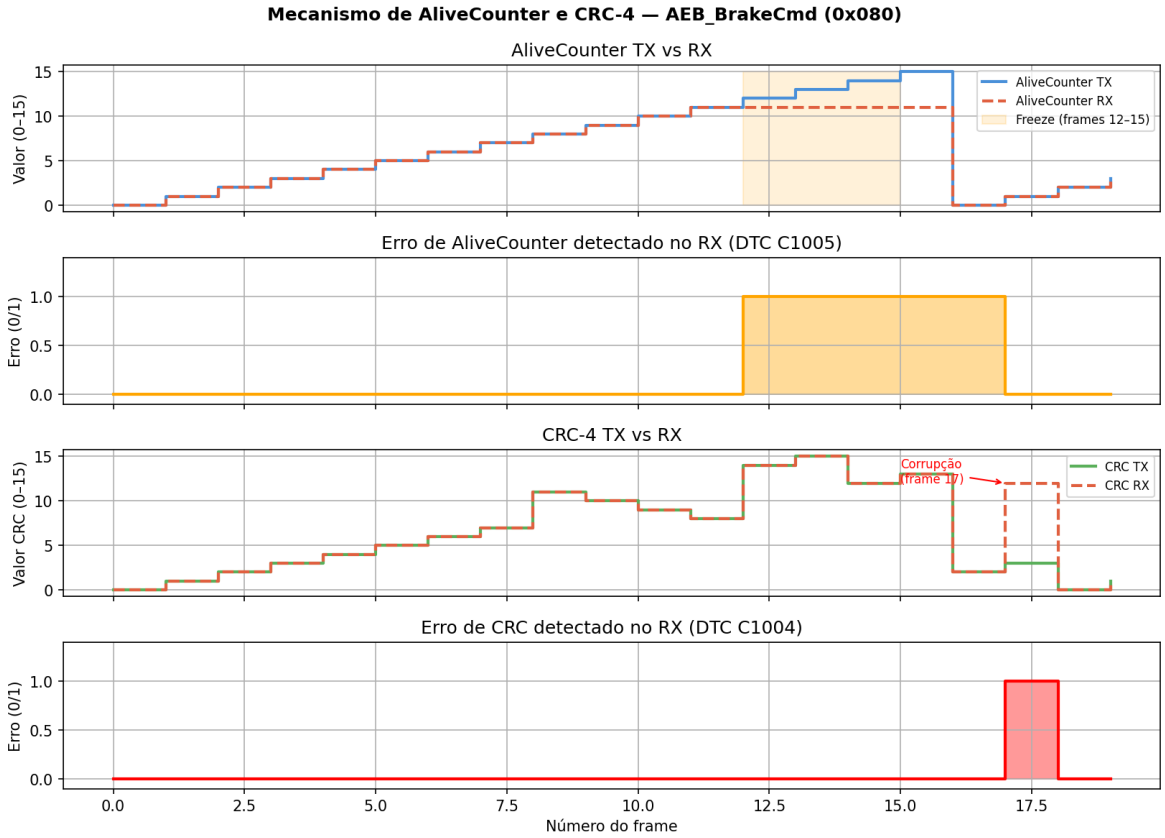

Figura 4: Simulação do AliveCounter e CRC-4: freeze entre frames 12–15 gera `alive_err=1` (DTC C1005); corrupção no frame 17 gera `crc_err=1` (DTC C1004).

6 CRC de 4 Bits

O campo `CRC` de 4 bits na mensagem `AEB_BrakeCmd` protege a integridade dos três primeiros bytes de payload. É calculado por XOR simples:

CRC-4 — AEB_BrakeCmd

$$\text{CRC} = (\text{alive} \oplus \text{brake_req} \oplus \text{brake_mode})\ \&\ \texttt{0x0F} \tag{4}$$

onde $\oplus$ denota XOR bit-a-bit e & `0x0F` extrai apenas os 4 bits menos significativos.

Esta função está definida em `aeb_controller_node.cpp`:

Listing 4: CRC-4 no bloco TX (MATLAB Function)

```
% b0=alive_counter, b1=brake_request, b2=brake_mode
crc = double(bitand( ...
    bitxor(bitxor(uint8(alive), uint8(bc_req_r)), uint8(bc_mode_r))
        , ...
    uint8(15)));  % mascara 0x0F
```

Nota Técnica

Embora o XOR de 4 bits não seja um CRC polinomial automotivo (como o CRC-8 SAE J1939), é suficiente para detectar erros de bit único e serve como mecanismo educacional representativo de proteção de mensagem ASIL-B.

7 Injeção de Falhas

O subsistema `Injecao_Falhas` implementa três tipos de falha controláveis via sinais de workspace:

Tipo	Descrição	DTC gerado
`fault_dropout`	Supressão completa do frame (`valid=0`)	C1010
`fault_crc`	Inversão de todos os 4 bits do CRC (XOR 0x0F)	C1004
`fault_alive`	Congelamento do AliveCounter (valor não incrementa)	C1005

Listing 5: Falha de CRC no subsistema Injecao_Falhas

```
if f_crc > 0
    crc_rx = double(bitxor(uint8(crc), uint8(15)));  % inverte 4 bits
else
    crc_rx = crc;
end
```

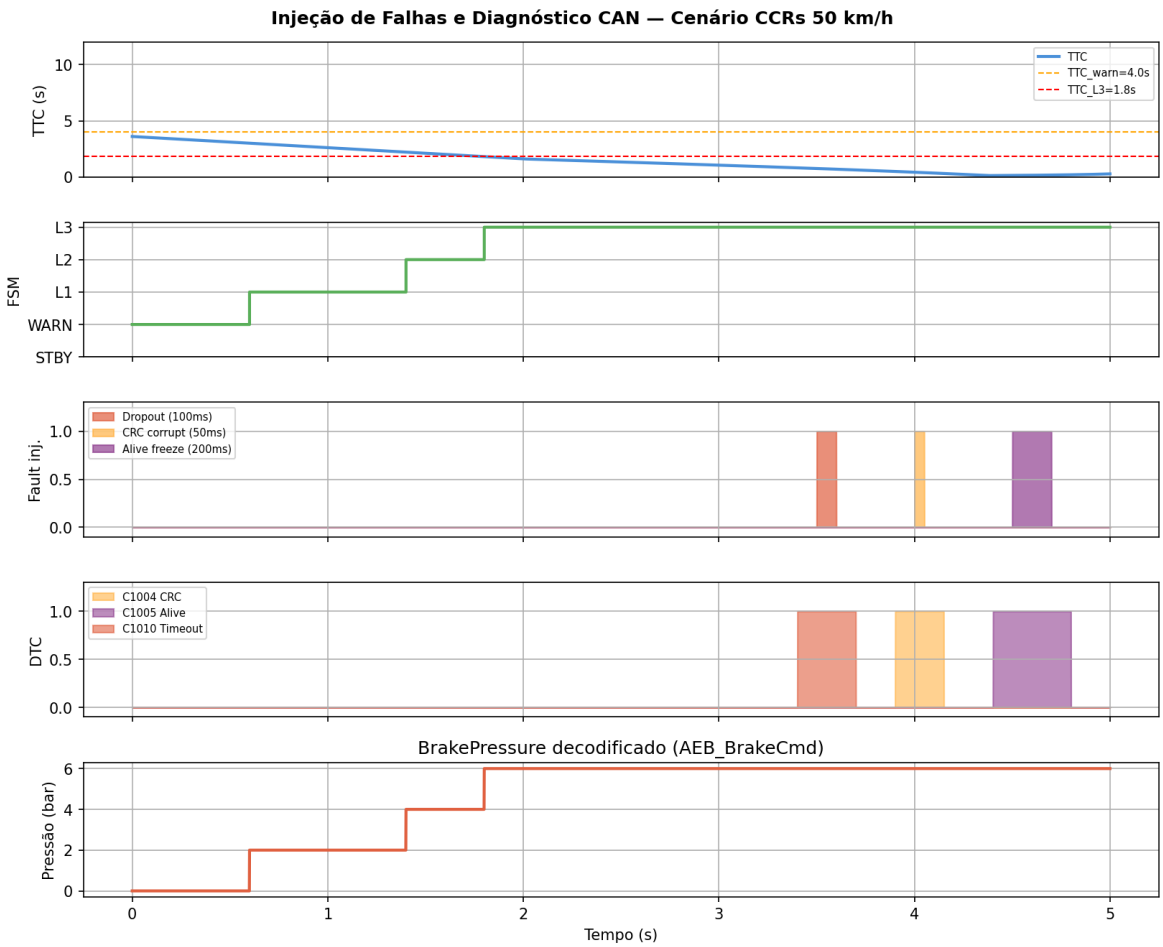

Figura 5: Cenário CCRs 50 km/h com injeção de falhas: dropout em $t = 3{,}5$ s, CRC em $t = 4{,}0$ s e alive freeze em $t = 4{,}5$ s. Os DTCs C1004, C1005 e C1010 são confirmados após debounce de 3 ciclos.

8 Diagnóstico CAN

O subsistema `Diagnosticos` implementa a lógica de debounce de DTCs conforme `SENSOR_FAULT_CYCLES=3` definido em `aeb_config.h`:

Listing 6: Debounce de DTC no MATLAB Function DTC_Logic

```
% Contador por tipo de falha (sobe/desce com histerese)
if crc_err,    cnt_crc    = min(cnt_crc + 1, 5);
else           cnt_crc    = max(cnt_crc - 1, 0); end
% ... idem para alive_err e timeout_err

% DTC confirmado apos 3 ciclos consecutivos
dtc_c1004 = double(cnt_crc    >= 3);  % Erro de CRC
dtc_c1005 = double(cnt_alive  >= 3);  % AliveCounter travado
dtc_c1010 = double(cnt_tmo    >= 3);  % Timeout de mensagem
```

DTC	Condição	Detecção	ISO 14229
C1004	CRC recebido $\neq$ CRC calculado	Imediato	ReadDTCInformation 0x19
C1005	AliveCounter travado >5 ciclos	3 ciclos	ReadDTCInformation 0x19
C1010	Timeout de mensagem >10 ms	3 ciclos	ReadDTCInformation 0x19

9 Pipeline do Modelo Simulink

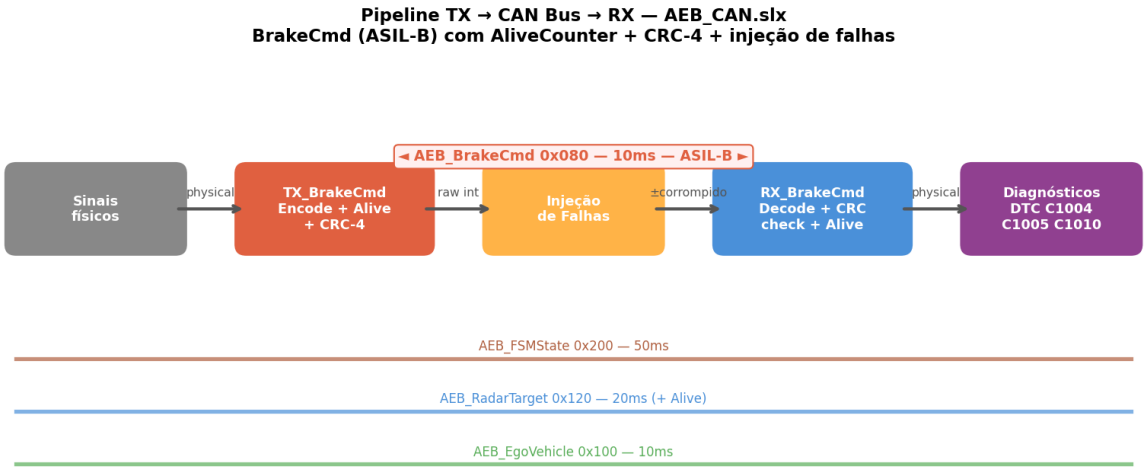

Figura 6: Pipeline do modelo `AEB_CAN.slx`: o caminho ASIL-B (`AEB_BrakeCmd`) inclui AliveCounter, CRC-4 e injeção de falhas; os demais caminhos usam apenas codificação DBC.

O modelo `AEB_CAN.slx` é composto por 12 subsistemas:

1. **TX_BrakeCmd** — codificação + AliveCounter + CRC-4 (10 ms, ASIL-B)
2. **TX_EgoVehicle** — codificação DBC (10 ms)
3. **TX_RadarTarget** — codificação + AliveCounter (20 ms)
4. **TX_FSMState** — codificação DBC (50 ms)
5. **TX_Alert** — codificação DBC (evento)
6. **Injecao_Falhas** — dropout, CRC, alive freeze
7. **RX_BrakeCmd** — decodificação + verificação CRC + alive + timeout
8. **RX_EgoVehicle** — decodificação DBC
9. **RX_RadarTarget** — decodificação + verificação alive
10. **RX_FSMState** — decodificação DBC
11. **RX_Alert** — decodificação DBC
12. **Diagnosticos** — debounce e confirmação de DTCs

10 Resultados de Simulação

10.1 Sinais decodificados — cenário nominal

O cenário padrão simula um evento CCRs a 50 km/h (13,89 m/s), com o AEB ativando frenagem a partir de $t \approx 2$ s quando o TTC cruza os limiares definidos em `aeb_config.h` ($\mathrm{TTC}_{\mathrm{warn}} = 4{,}0$ s, $\mathrm{TTC}_{L1} = 3{,}0$ s, $\mathrm{TTC}_{L2} = 2{,}2$ s, $\mathrm{TTC}_{L3} = 1{,}8$ s).

Nota Técnica

A comparação entre os sinais TX e RX decodificados verifica a corretude dos fatores e *offsets* do DBC. Para todos os sinais sem injeção de falha, TX = RX a menos de erros de quantização $\leq$ fator/2.

10.2 Parâmetros DBC consolidados

Parâmetros DBC — aeb_system.dbc (500 kbps)

Mensagem	CAN ID	Período	Bytes	Nó TX	Sinal chave	Factor / Offset
AEB_BrakeCmd	0x080	10 ms	4	AEB_Controller	BrakePressure [bar]	0,1 / 0 (15b)
AEB_EgoVehicle	0x100	10 ms	8	Vehicle_ECU	VehicleSpeed [m/s]	0,01 / 0 (16b)
AEB_RadarTarget	0x120	20 ms	8	Radar_ECU	TargetDistance [m]	0,01 / 0 (16b)
AEB_FSMState	0x200	50 ms	4	AEB_Controller	TTCThreshold [s]	0,1 / 0 (8b)
AEB_Alert	0x300	evento	2	AEB_Controller	BuzzerCmd	1 / 0 (3b)

Figura 7: Tabela de parâmetros DBC das 5 mensagens CAN do sistema AEB.

10.3 Uso

Listing 7: Uso do modelo AEB_CAN.slx

```
% 1. Criar o modelo
run('AEB_CAN_build.m')

% 2. Cenario nominal
out = sim('AEB_CAN');
AEB_CAN_plot(out)

% 3. Injetar falha de CRC
AEB_CAN_scenarios('fault_crc')
out = sim('AEB_CAN');
AEB_CAN_plot(out)

```

```
% 4. Injetar dropout + alive freeze combinados
AEB_CAN_scenarios('fault_combined')
out = sim('AEB_CAN');
```

11 Sincronização v_target_cached_

Um problema sutil de temporização assíncrona ocorre porque `AEB_EgoVehicle` chega a 10 ms e `AEB_RadarTarget` a 20 ms. Se o controlador calculasse $v_{\text{target}} = v_{\text{ego}} - v_{\text{rel}}$ no ciclo de 10 ms usando o `relative_speed` mais recente (com até 20 ms de atraso) e o `vehicle_speed` atual, o resultado poderia ser inconsistente durante acelerações rápidas:

Solução: v_target_cached_

$$v_{\text{target}} = v_{\text{ego}}\big|_{\text{@radar_callback}} - v_{\text{rel}}\big|_{\text{@radar_callback}} \tag{5}$$

v_{target} é calculado **dentro do callback do radar**, quando v_{ego} e v_{rel} são contemporâneos. No ciclo de 10 ms, apenas `v_target_cached_` é passado para `aeb_cycle_10ms()` — nunca o `relative_speed` diretamente.

12 Conclusão

O modelo `AEB_CAN.slx` demonstra os princípios fundamentais do barramento CAN no sistema AEB:

- **Codificação DBC**: conversão física ↔ raw com fator, offset e saturação correta para todos os 5 sinais-chave.
- **AliveCounter**: contador rolante de 4 bits módulo 16 presente em `AEB_BrakeCmd` e `AEB_RadarTarget`; verificação instantânea no receptor.
- **CRC-4**: XOR dos campos `alive`, `brake_req` e `brake_mode`, mascarado em 4 bits — detecta erros de bit único.
- **Injeção de falhas**: três tipos controláveis (dropout, CRC corrupto, alive freeze) com janelas de tempo configuráveis.
- **Diagnóstico**: debounce de 3 ciclos para confirmação dos DTCs C1004, C1005 e C1010, compatível com ISO 14229.

Nota Técnica

Vehicle Network Toolbox: com o toolbox disponível, os blocos nativos `CAN Pack` / `CAN Unpack` podem importar `aeb_system.dbc` diretamente, substituindo os blocos MATLAB Function de codificação. A arquitetura de subsistemas e a lógica de AliveCounter/CRC permanecem idênticas.